

ERP Monorepo Structure and Git Collaboration Guide

1. Repository Structure

```
erp-system/
├─ README.md
├─ .gitignore
├─ .editorconfig
├─ .gitattributes
├─ docker-compose.yml
├─ Makefile
├─ package.json # optional root scripts if using workspace
├─ tooling
├─ docs/
│   └─ architecture/
│       ├── erp-master-architecture.md
│       ├── service-boundaries.md
│       ├── event-contracts.md
│       ├── frontend-architecture.md
│       └─ database-strategy.md
│   └─ modules/
│       ├── hrm-blueprint.md
│       ├── prms-blueprint.md
│       ├── mms-blueprint.md
│       ├── crm-blueprint.md
│       └─ fms-blueprint.md
│   └─ api/
│       ├── hrm-openapi.yaml
│       ├── prms-openapi.yaml
│       ├── mms-openapi.yaml
│       ├── crm-openapi.yaml
│       └─ fms-openapi.yaml
│   └─ frontend/
│       ├── route-ownership.md
│       ├── shared-components.md
│       └─ ui-conventions.md
│   └─ workflows/
│       └─ git-strategy.md
```

```

├── pr-template.md
├── branching-model.md
├── release-checklist.md
├── infrastructure/
│   ├── docker/
│   │   ├── frontend.Dockerfile
│   │   ├── hrm-service.Dockerfile
│   │   ├── prms-service.Dockerfile
│   │   ├── mms-service.Dockerfile
│   │   ├── crm-service.Dockerfile
│   │   └── fms-service.Dockerfile
│   ├── postgres/
│   │   └── init/
│   │       ├── 01-create-databases.sql
│   │       └── 02-create-schemas.sql
│   ├── rabbitmq/
│   │   └── definitions.json
│   ├── gateway/                                # optional if later you add API gateway
│   └── scripts/
│       ├── start-dev.sh
│       ├── stop-dev.sh
│       ├── reset-db.sh
│       └── run-migrations.sh
├── frontend/
│   ├── package.json
│   ├── next.config.ts
│   ├── tsconfig.json
│   ├── middleware.ts
│   ├── public/
│   └── src/
│       ├── app/
│       │   ├── (dashboard)/
│       │   │   ├── layout.tsx
│       │   │   ├── page.tsx
│       │   │   └── hrm/
│       │   │       ├── employees/
│       │   │       │   ├── page.tsx
│       │   │       │   ├── new/page.tsx
│       │   │       │   └── [id]/page.tsx
│       │   │       ├── departments/
│       │   │       └── attendance/

```

```

└─ leave-requests/

└─ prms/
  └─ suppliers/
  └─ purchase-requests/
  └─ purchase-orders/

└─ mms/
  └─ items/
  └─ warehouses/
  └─ stock-movements/
  └─ goods-receipts/

└─ crm/
  └─ customers/
  └─ leads/
  └─ quotations/
  └─ sales-orders/

└─ fms/
  └─ chart-of-accounts/
  └─ journals/
  └─ invoices/
  └─ payments/
  └─ reports/

└─ login/
  └─ page.tsx

└─ unauthorized/
  └─ page.tsx

└─ features/
  └─ hrm/
    └─ api/
    └─ components/
    └─ hooks/
    └─ schemas/
    └─ types/
    └─ utils/

  └─ prms/
    └─ api/
    └─ components/
    └─ hooks/
    └─ schemas/
    └─ types/

```

```

└─ utils/

└─ mms/
  └─ api/
  └─ components/
  └─ hooks/
  └─ schemas/
  └─ types/
  └─ utils/

└─ crm/
  └─ api/
  └─ components/
  └─ hooks/
  └─ schemas/
  └─ types/
  └─ utils/

└─ fms/
  └─ api/
  └─ components/
  └─ hooks/
  └─ schemas/
  └─ types/
  └─ utils/

└─ shared/
  └─ api/
    └─ http-client.ts
    └─ auth-client.ts
    └─ query-client.ts

  └─ components/
    └─ data-table/
    └─ form/
    └─ dialog/
    └─ status-badge/
    └─ page-header/
    └─ layout/

  └─ config/
    └─ nav-config.ts
    └─ permissions.ts
    └─ env.ts

└─ hooks/

```



```

├── hrms-events.md
├── prms-events.md
├── mms-events.md
├── crm-events.md
├── fms-events.md
├── api/
│   ├── shared-dtos.md
│   └── integration-contracts.md
├── examples/
│   ├── purchase-order-approved.json
│   ├── stock-received.json
│   └── invoice-generated.json
├── scripts/
│   ├── dev/
│   ├── ci/
│   └── seed/
├── .github/
│   ├── pull_request_template.md
│   └── workflows/
│       ├── frontend-ci.yml
│       ├── hrms-ci.yml
│       ├── prms-ci.yml
│       ├── mms-ci.yml
│       ├── crm-ci.yml
│       └── fms-ci.yml

```

2. Why this structure works

This structure works because it separates the ERP into **clear ownership zones**:

```
frontend/
```

Contains the **single shared Next.js app**.

```
services/
```

Contains the **5 Spring Boot** microservices.

docs/

Contains architecture, module blueprints, API docs, ownership rules, event contracts, and workflow docs.

infrastructure/

Contains Docker, Postgres, RabbitMQ, local dev setup, and startup scripts.

shared-contracts/

Contains **cross-team agreements**: event payloads, integration request/response examples, and shared API contracts.

This is important because in microservices, the code is only half the system. The other half is **contracts and rules**.

3. Frontend Folder Ownership Rules

Shared frontend ownership zones

These folders are **shared** and must be changed carefully:

- `src/shared/components/`
- `src/shared/api/`
- `src/shared/config/`
- `src/providers/`
- `src/app/(dashboard)/layout.tsx`
- `src/app/login/`
- `middleware.ts`

These should require **review from the frontend lead or project lead**.

Team-owned frontend zones

Each module team owns only its own feature area:

- HRM team → `src/features/hrm/` and `src/app/(dashboard)/hrm/`
- PRMS team → `src/features/prms/` and `src/app/(dashboard)/prms/`
- MMS team → `src/features/mms/` and `src/app/(dashboard)/mms/`
- CRM team → `src/features/crm/` and `src/app/(dashboard)/crm/`
- FMS team → `src/features/fms/` and `src/app/(dashboard)/fms/`

If a PR from the HRM team changes `src/features/crm`, that should be a red flag unless it's explicitly coordinated.

4. Backend Folder Ownership Rules

Each backend team owns only its service under `services/`.

HRM team owns

- `services/hrm-service/**`

PRMS team owns

- `services/prms-service/**`

MMS team owns

- `services/mms-service/**`

CRM team owns

- `services/crm-service/**`

FMS team owns

- `services/fms-service/**`

A team must **never** change another team's schema, entity, business logic, or controller directly unless it was explicitly agreed in a shared integration task.

5. Internal Spring Boot Service Structure

Each service should use **feature-first structure**, not a giant controller/service/repository dumping ground.

Example: HRM service

```
hrm-service/  
└─ src/main/java/com/company/hrm/  
    ├── shared/  
    │   ├── config/  
    │   └── security/
```

```
|
| | exception/
| | audit/
| | └─ util/
|
| └─ department/
|   | controller/
|   | dto/
|   | entity/
|   | repository/
|   | service/
|   | mapper/
|   | └─ event/
|
| └─ employee/
|   | controller/
|   | dto/
|   | entity/
|   | repository/
|   | service/
|   | mapper/
|   | └─ event/
|
| └─ attendance/
| └─ leave/
| └─ payrollsupport/
```

Why feature-first instead of layer first?

Because if you use this:

```
controller/
service/
repository/
entity/
dto/
```

for the entire service, all team end up editing the same folders constantly. you will face merge conflict .

Feature first keeps things modular:

- one team works in `employee/`
- another in `attendance/`
- another in `leave/`

Much less collision.

6. Git Strategy for 5 teams

how to stop Git from turning into civil war ?

Use this branching model:

long lived branches

- `main` → stable demo ready branch
- `develop` → integration branch for all completed features

Short lived branches

Each task gets its own feature branch:

```
feature/hrm-employee-crud  
feature/hrm-leave-approval  
feature/prms-purchase-request  
feature/mms-item-master  
feature/crm-customer-management  
feature/fms-journal-entry  
fix/hrm-attendance-validation  
refactor/frontend-shared-table
```

7. Branch Naming Convention

Use this format:

```
feature/<module>-<feature-name>  
fix/<module>-<issue-name>  
refactor/<scope>-<name>  
docs/<topic>
```

Examples

- `feature/hrm-department-management`
- `feature/prms-supplier-crud`
- `feature/mms-goods-receipt`
- `feature/crm-sales-order-page`

- feature/fms-payment-posting
- fix/frontend-nav-permissions
- docs/event-contracts

8. Mandatory Git Workflow

Every team should follow this exact workflow.

Step 1: Pull latest develop

```
git checkout develop
git pull origin develop
```

Step 2: Create a fresh feature branch from develop

```
git checkout -b feature/hrm-employee-crud
```

Step 3: Work only on your assigned files

Do not “clean up unrelated code” in the same PR. That’s how small features become cursed.

Step 4: Commit small, meaningful commits

```
git add .
git commit -m "feat(hrm): add employee create endpoint"
git commit -m "feat(hrm): add employee list page"
git commit -m "test(hrm): add employee service tests"
```

Step 5: Push branch

```
git push origin feature/hrm-employee-crud
```

Step 6: Open PR into develop

Not into `main`.

Step 7: Resolve review comments

Then merge into `develop`.

9. How to Reduce Git Conflicts Before They Happen

Rule 1: Strict file ownership

Each team owns a module folder.

Each student within a team owns specific features.

Example inside HRM team

- student 1 → `employee/`
- student 2 → `department/position/`
- student 3 → `attendance/`
- student 4 → `leave/`
- student 5 → frontend HRM pages + testing support

If two student are editing the same file every day, the task split is wrong.

Rule 2: One person owns shared frontend shell changes

Files like:

- `layout.tsx`
- `nav-config.ts`
- auth provider
- sidebar config
- shared table/form components

should not be edited by all 25 people .

Assign **1 or 2 frontend maintainers** for shared frontend infrastructure

Rule 3: Avoid giant “files”

These files create conflict magnets:

- one huge `routes.ts`
- one huge `permissions.ts`

- one massive `constants.ts`
- one giant `Sidebar.tsx`
- one giant `ApiService.ts`

Split them.

Better approach

Instead of one giant nav config:

```
// bad
src/shared/config/nav-config.ts
```

Use:

```
src/shared/config/navigation/
├─ hrm-nav.ts
├─ prms-nav.ts
├─ mms-nav.ts
├─ crm-nav.ts
├─ fms-nav.ts
└─ index.ts
```

Then merge them in one place.

Same trick for permissions, route registration, and API configs.

Rule 4: PRs must be small

A PR should ideally be **one feature or one vertical slice**, not “I touched 97 files because vibes.”

Good PR examples

- Employee CRUD backend
- Leave approval workflow backend
- HRM employee list page
- PRMS supplier table + create form

Bad PR examples

- “finished hrm”
- “frontend fixes”
- “all backend changes”
- “refactor everything”

That kind of PR is how review quality dies.

Rule 5: Rebase or sync frequently

If someone works on a branch for 10 days without syncing `develop`, they're basically crafting a conflict bomb.

At minimum, interns should sync with `develop` **daily** or every time before pushing major work.

Example

```
git checkout develop
git pull origin develop
git checkout feature/hrm-employee-crud
git merge develop
```

or if your team is comfortable with rebase:

```
git checkout develop
git pull origin develop
git checkout feature/hrm-employee-crud
git rebase develop
```

For interns, **merge from develop is usually safer than forcing rebase discipline** unless they already know Git well.

10. Conflict Resolution Workflow

When a conflict happens, don't freestyle it.

Standard conflict process

1. Pull latest develop into your feature branch

```
git checkout develop
git pull origin develop
git checkout feature/hrm-employee-crud
git merge develop
```

2. Open conflicted files and inspect carefully

Look for:

```
<<<<<< HEAD
your code
=====
incoming code
>>>>>> develop
```

3. Do not just delete markers randomly

Understand:

- what changed in your branch
- what changed in develop
- whether both changes should stay

4. Run the project after resolving

For frontend:

```
npm run lint
npm run build
```

For backend:

```
./gradlew test
```

5. Commit the merge resolution

```
git add .
git commit -m "merge develop into feature/hrm-employee-crud and resolve
conflicts"
```

11. Rules for Shared Files

Some files are **high-conflict files** and should have stricter rules.

High-conflict frontend files

- `src/app/(dashboard)/layout.tsx`
- `src/shared/config/nav-config.ts`
- `src/shared/config/permissions.ts`
- `src/shared/api/http-client.ts`
- `middleware.ts`

High-conflict backend files

- root `application.yml`
- shared security config
- RabbitMQ exchange/queue config if centralized
- global exception classes
- common enum files if everybody keeps dumping things there

Rule

Any PR that touches shared files must:

- mention it in PR description
- explain why it was needed
- be reviewed by module lead or project lead

12. Recommended PR Template

Every PR should answer these questions:

Pull Request Template

- **Module:** HRM / PRMS / MMS / CRM / FMS / Frontend Shared
- **Feature:** what is implemented?
- **Type:** feature / fix / refactor / docs
- **Backend changes:** yes/no
- **Frontend changes:** yes/no
- **DB migration included:** yes/no
- **Event contract changed:** yes/no
- **Shared file touched:** yes/no
- **Screenshots included:** yes/no
- **How to test:** steps
- **Known limitations:** if any

This alone will save you from “what exactly did this branch do?” conversations.

13. Recommended Git Rules for the Internship

Rule 1

Nobody pushes directly to `main`.

Rule 2

Nobody merges their own PR without review.

Rule 3

Every task must have a branch.

Rule 4

Every PR must target `develop`.

Rule 5

Every team syncs with `develop` frequently.

Rule 6

Every shared-file change must be explicitly called out.

Rule 7

If two interns need the same file, stop and redesign ownership before they both start coding.

Rule 8

Do not mix unrelated changes in one branch.

Rule 9

Do not rename/move huge folders casually mid-project unless planned. That's Git-conflict napalm.

Rule 10

Finish one feature branch and merge it before starting five half-dead branches in parallel.

14. Team Level Workflow to Prevent Collisions

Each team should run a very simple board with statuses:

- Backlog
- Ready
- In Progress
- PR Review
- Blocked
- Done

And every task card should show:

- owner
- service/module
- frontend/backend
- affected folders
- dependency on another team

this sounds boring but it answers the most important question in a 30 person repo:

“Who is touching what right now?”

If nobody knows that, conflicts are guaranteed.

16. Final Recommendation

Use:

- 1 monorepo
- 1 shared Next.js frontend
- 5 Spring Boot services under `/services`
- feature-first package structure inside each service
- module-based feature folders in frontend
- strict branch-per-task Git workflow
- shared-file protection rules
- PR review into `develop`, never direct to `main`
- daily sync with `develop`
- small PRs, not mega PRs